Lecture 1 — The World Finals Mindset

> World Finals Level · Prerequisites: Year Curriculum, Regionals and Nationals tracks (in particular *regional-01 Contest anatomy*, *nationals-01 Training system*, *week-35 Contest strategy*) · Estimated time: 90 minutes lecture + one full 5-hour virtual contest

1. Overview

The ICPC World Finals (WF) is the hardest team contest most students will ever take part in. This track teaches the techniques that separate medal teams from the rest of the field: polynomial algebra, LP and general matching, link-cut trees, offline divide and conquer, robust geometry, advanced combinatorics, strings and DP. Techniques alone do not win medals, though. At WF the techniques are hidden behind modelling, the set is long and unsorted, and you have one computer for three people.

This first lecture is about the contest itself:

1. the **format**, stated precisely, separating the stable rules from what changes year to year;
2. the **archetypes** of WF problems, with real examples from past finals;
3. a **5-hour strategy** for a three-person, one-computer team;
4. how to **train with past finals** and run useful post-mortems.

The rest of the track (Lectures 2–9) is technique-heavy. Lecture 10 brings everything together: building the team reference document and running a full simulation.

2. The format

2.1 Stable facts

Item	World Finals
Team	three contestants (plus a coach who is not in the contest room)
Machine	one computer per team
Duration	five hours
Problem count	usually around 11–13; it varies by year
Problem order	lettered A, B, C, … usually in alphabetical order of title, **not** by difficulty
Reference material	a printed team reference document (historically limited to 25 single-sided pages)
Ranking	number of problems solved, then total penalty time

Item	World Finals
Scoreboard	public during the contest, frozen for the final hour

Things that do change, and that you must check in the current year's rules and environment documents: the list of supported languages (recently C, C++, Java, Kotlin and Python 3), the compiler versions and flags, the exact page limit and format of the reference document, and the eligibility rules (contestants are limited in how many World Finals they can attend).

2.2 Penalty time

For each **solved** problem, its penalty is the number of minutes from the start of the contest until the accepted submission, plus 20 minutes for every rejected submission on that problem before the accepted one. A team's penalty time is the sum over solved problems. Rejected submissions on problems that are never solved cost nothing.

Two consequences follow, and every team member should know them by heart.

- **Solving one more problem beats any amount of penalty.** A team that solves 8 problems in 299 minutes each beats a team that solved 7 problems in the first hour. When you are choosing between polishing a solved problem's time and starting a new one, there is no contest: a new solve always wins.
- **Penalty matters among teams with the same count**, and at WF the medal boundary often falls between teams with the same number of solves. Getting the easies in quickly and without wrong submissions is real ranking value.

2.3 Medals and the freeze

Traditionally the top four teams receive gold medals, the next four silver and the next four bronze, and the rules allow the judges to award extra medals. During the last hour the public scoreboard stops updating. You still see your own results, but not other teams'. The final standings are revealed afterwards. Practically, the freeze means that the strategy decisions you make in hour five are made without knowing where you stand. Decide on the endgame *before* the freeze (Section 4.4).

3. Archetypes of WF problems

Experienced WF teams classify problems within a minute of reading them. The labels below are not official; they are a practical vocabulary. Each archetype is illustrated with real past WF problems. The titles are exact, and all of them can be practised on the Kattis open archive, which hosts past World Finals sets.

3.1 The "obvious once seen" easy

Every set has two or three problems that most teams solve. They are short and usually reduce to a standard tool on a small model.

- **WF 2017 *Secret Chamber at Mount Rushmore*.** You are given letter translation rules and pairs of words, and asked whether each word can be turned into the other. Model the letters as a 26-vertex directed graph and compute reachability (Floyd–Warshall on 26 nodes). The only traps are pairs whose lengths differ.
- **WF 2016 *Ceiling Function*.** Insert sequences into binary search trees and count how many distinct tree *shapes* arise. Build the trees and compare canonical shape strings.
- **WF 2019 *Circular DNA*.** Choose a cut point of a circular sequence to maximise the number of properly nested gene markers. This is a prefix-balance argument over all rotations.

Rule: the person who reads an easy problem and sees the solution **codes it immediately**, unless the machine is busy with another easy. Easies are a race, and every minute of delay is a minute of penalty.

3.2 Binary search on a hidden quantity

WF 2017 *Need for Speed*. A speedometer reads `s_i` while the true speed is `s_i + c` for an unknown constant `c`. Given segment lengths and the total time `t`, find `c`. The total time `Σ d_i / (s_i + c)` is strictly decreasing in `c` on the domain `c > −min s_i`, so you bisect on `c`. The pitfalls are typical of this archetype: the domain boundary (true speeds must be positive), choosing an upper bound, and using a fixed number of iterations (100) instead of an `eps` loop with doubles.

3.3 A classical algorithm in disguise

Most medium-tier WF problems are solved by a technique that every finalist knows. The difficulty is seeing which one applies.

- **WF 2016 *Branch Assignment*.** After two Dijkstra runs (on the graph and on its reverse), each branch has a weight. You then group branches into `s` groups, and the cost of a group is (size − 1) × (sum of its weights). Sort the weights and the optimal groups become contiguous, which gives a partition DP that divide-and-conquer optimisation speeds up (*nationals-03*).
- **WF 2017 *Money for Nothing*.** Choose a producer `(p_x, p_y)` and a consumer `(q_x, q_y)` to maximise `(q_x − p_x)(q_y − p_y)`. After you discard dominated points, the optimal partner index is monotone, which again gives a divide-and-conquer solution.
- **WF 2015 *Catering*.** Serve a sequence of requests with at most `k` teams that move between locations at a cost. This is min-cost flow (*nationals-05*, and Lecture 3 for bounds).

- **WF 2015 *Tile Cutting*.** The number of ways to tile area `a` is a convolution of divisor counts, and answering for all `a` up to $5 \cdot 10^5$ needs FFT/NTT (Lecture 2).

3.4 Geometry

There is a geometry problem in essentially every set, and often more than one. Some are hard because of a real idea, others because of cases and precision.

- **WF 2017 *Airport Construction*.** Find the longest straight segment that fits inside a simple polygon. The candidate segments pass through two polygon vertices. The difficulty is handling collinear and touching configurations correctly.
- **WF 2018 *Panda Preserve*.** Find the smallest radius such that circles of that radius around the polygon's vertices cover the polygon. The structure is a Voronoi diagram restricted to the polygon (Lecture 6).

Decide on a precision policy early: exact integer predicates (`long long`, `__int128`) wherever the input is integral, and `long double` with a single, documented epsilon where it is not.

3.5 The long implementation

- **WF 2019 *Karel the Robot*.** Interpret a small robot language with procedures, conditionals and loops, and detect non-termination. Memoise `(procedure, state)` pairs.
- **WF 2017 *Visual Python++*.** Match top-left corners to bottom-right corners so that the rectangles form a properly nested family. You need a sweep with an ordered set plus a validation pass.

These problems are rarely conceptually hard, but they eat machine time. Write them on paper first and give them to your most careful implementer for a long, uninterrupted block.

3.6 The deep problem

These problems need a technique from this track combined with a real insight.

- **WF 2018 *Conquer the World*.** Move armies on a weighted tree to meet demands at minimum cost. It is min-cost flow on a tree, and the accepted solutions use greedy-with-regret plus mergeable heaps (small-to-large or leftist heaps).
- **WF 2017 *Son of Pipe Stream*.** Two fluids share a network with capacity constraints, and you maximise a concave function of the two flow values. This combines max-flow structure with continuous optimisation.

Medal teams typically solve the easy and medium tiers cleanly and then **one or two** of the deep problems. Solving a single deep problem that the teams around you do not solve is exactly how the medal boundary moves.

4. A 5-hour strategy for three people and one computer

4.1 The first 15 minutes

Split the set three ways by letter (A–D, E–H, I–M). Everyone reads their share completely, including the limits and the sample explanations. Each reader writes on the shared problem sheet a one-line summary of every problem, an estimated tier, and the solution idea if one is obvious. Meanwhile the designated driver types the template, the stress-test script and a quick "hello world" submission to check the environment. Do this only if your team's plan includes a test submission and the rules allow it.

4.2 Hours one and two: the easy and medium tiers

- Code the easies in order of *certainty*, not of reading order.
- While one person is at the machine, the other two do not sit idle. One thinks about the next problem; the other writes pseudo-code or test cases for the problem after that.
- Use the live scoreboard as a difficulty oracle. If many teams have solved a problem that you rated hard, re-read it: you have probably missed a simplification. If nobody has touched a problem after two hours, be careful before investing in it.

4.3 Hours two to four: committing

By the 2:30 mark the team should agree on a **short list**: the one or two remaining problems you will finish. Pair two people on the hardest viable problem: one proves the idea and the other attacks it with counterexamples. Problems that nobody has a plan for at this point should not be started unless the scoreboard suggests they are easier than they look.

4.4 The last hour

Decide the endgame before 4:00, when the scoreboard freezes:

- every unfinished problem gets an explicit owner and a stop time;
- after about 4:30, no new problem is started unless it is very short;
- problems with a wrong answer get a fresh pair of eyes and a stress test before they get a rewrite.

4.5 Keyboard discipline

The computer is the scarce resource. Three rules follow.

1. **Never think at the keyboard.** Arrive with pseudo-code and edge cases already written down.

2. **Debug on paper.** If your submission fails, print the code (if printing is available) and give the machine to someone else while you read it.
3. **Make hand-offs explicit.** Say "I need about 15 minutes for C", and the person waiting prepares in parallel.

4.6 Roles

The three roles below work well for most teams. Roles rotate per problem; what must be fixed in advance is who has the final word when two people want the machine at the same time.

Role	Main responsibilities
Driver	fastest clean coder; types most easies and long implementations
Thinker	owns the hardest viable problem; proofs, reductions, pseudo-code
Verifier	reads all statements carefully; edge cases, brute forces, generators, stress tests

5. Estimating difficulty from a statement

Constraints are the most reliable hint you get.

- $n \le 20$ suggests subset DP or meet-in-the-middle. $n \le 500$ suggests a cubic algorithm (Hungarian, blossom, Floyd–Warshall).
- An odd pair of limits, such as $n \le 10^5$ with $k \le 50$, tells you which parameter can appear in a factor or an exponent.
- Output modulo 998244353 strongly suggests NTT-friendly polynomial work (Lecture 2).
- Real-valued output with 10^{-6} tolerance suggests binary search, geometry or LP (Lecture 3).
- "Print any valid answer" suggests construction or matching.
- Huge limits with a small, structured input (for example $k \le 10^{18}$ for a recurrence) suggest matrix power, Berlekamp–Massey, or a closed form.

6. Reference implementation: stress testing

Every finalist team should be able to type a stress test from memory in under a minute. The pieces are a **seeded** generator, a brute-force solution, the fast solution, and a shell loop.

```cpp
#include <bits/stdc++.h>
using namespace std;
// Usage: ./gen SEED N   -> random tree with N vertices and random weights
int main(int argc, char** argv) {
    if (argc < 3) { cerr << "usage: gen seed n\n"; return 1; }
    mt19937 rng(atoi(argv[1]));
    int n = atoi(argv[2]);
    auto rnd = [&](int a, int b) { return (int)(rng() % (b - a + 1)) + a; };
    vector<int> perm(n);
```

```
    iota(perm.begin(), perm.end(), 1);
    shuffle(perm.begin(), perm.end(), rng);     // hide the parent < child structure
    cout << n << "\n";
    for (int i = 1; i < n; i++)
        cout << perm[i] << " " << perm[rnd(0, i - 1)] << " " << rnd(1, 10) << "\n";
}
```

```
g++ -std=c++17 -O2 -o gen gen.cpp
g++ -std=c++17 -O2 -o fast fast.cpp
g++ -std=c++17 -O2 -o brute brute.cpp
for i in $(seq 1 1000); do
  ./gen $i 8 > in
  ./fast < in > out1
  ./brute < in > out2
  if ! cmp -s out1 out2; then echo "diff on seed $i"; cat in; break; fi
done
```

Three details matter:

- The generator takes the seed from the command line, so a failing case can be reproduced with `./gen 417 8`.
- The tree generator shuffles labels. Without the shuffle, every parent has a smaller label than its child, which hides bugs in code that assumes a particular rooting.
- Start with tiny `n` (5–8). Small counterexamples are the ones you can debug by hand.

For problems with many valid answers, replace `cmp` with a checker program that validates `out1` against the input.

7. Training with past finals

7.1 Simulations

Run a full past WF set as a five-hour virtual contest: three people, one computer, printed statements, and only your team reference document. Do this every one to two weeks during this track. Between simulations, work through the technique lectures and Codeforces Div1 D–F / AtCoder AGC problems on those techniques.

7.2 Upsolving

- Upsolve **every** problem that at least a handful of teams solved in the real contest. These are the problems you must be able to solve to medal.
- For the deep problems, read the judges' solution sketch or the post-contest analysis. Write down the technique and the insight, even if you do not implement it.
- Keep a log with one row per problem: archetype, technique, the key insight, and *why* your team missed it (wrong model, implementation too slow, never read it, gave up too early).

7.3 Post-mortem questions

1. What did we solve, and at what times? Where did the gaps between solves come from?
2. Which wrong submissions did we make, and what was the root cause of each (overflow, edge case, misread statement)?
3. Which problem did we abandon, and was abandoning it correct?
4. Which problem would a top team have started that we did not?
5. Which notebook entry did we need but not have?

Every simulation should produce at least one change to the notebook and one change to the team's procedure.

8. Common mistakes

- **Reading in letter order and coding in letter order.** The set is not sorted by difficulty, so triage first.
- **Two people at the keyboard.** One types; the other reads the printout or prepares the next problem.
- **Falling in love with a problem.** Set a stop time when you start. When it passes, hand the problem to a teammate or park it.
- **Ignoring the scoreboard.** In the first four hours it is the best difficulty estimate you have.
- **Over-trusting the scoreboard.** Strong teams sometimes start with long problems. A problem with zero solves at 1:00 is not necessarily hard.
- **Submitting without testing the samples and one extreme case.** At WF, every wrong submission on a problem you later solve costs 20 minutes.
- **Starting a new problem at 4:45** when a nearly-correct problem needs a fresh pair of eyes.
- **Not knowing the environment.** Practise with the same compiler flags, editor and language versions the finals will use.

9. Practice problems

All World Finals problems below are on the Kattis open archive (search by title) and in the official ICPC problem archive.

1. **WF 2017 — *Secret Chamber at Mount Rushmore*** (easy: reachability on a 26-node graph).
2. **WF 2017 — *Need for Speed*** (binary search on the hidden parameter, precision).
3. **WF 2016 — *Ceiling Function*** (easy: BST shapes).
4. **WF 2016 — *Swap Space*** (exchange-argument greedy; review *regional-03*).
5. **WF 2016 — *Branch Assignment*** (Dijkstra plus D&C DP optimisation; review *nationals-03*).

6. **WF 2017 — *Money for Nothing*** (monotone divide and conquer).
7. **WF 2015 — *Tile Cutting*** (FFT/NTT; preview of Lecture 2).
8. **WF 2015 — *Catering*** (min-cost flow; preview of Lecture 3).
9. **WF 2019 — *Karel the Robot*** (long implementation with memoisation).
10. **Codeforces 1305F — *Kuroni and the Punishment*** (randomisation plus a structural insight): https://codeforces.com/problemset/problem/1305/F
11. **Full simulation:** any complete past WF set as a five-hour virtual contest.

10. Further reading

- ICPC World Finals problem archive and past results: https://icpc.global/worldfinals/problems
- Kattis open archive (search for "World Finals" in the problem sources): https://open.kattis.com/
- The current ICPC rules page on https://icpc.global/ for the authoritative format, eligibility and reference-document rules.
- KACTL, KTH's team reference document, as an example of a mature WF notebook: https://github.com/kth-competitive-programming/kactl
- Lecture 10 of this track: building your own team reference document and running a full simulation.